MODALIDADE RESUMO EXPANDIDO (REX) – GRADUAÇÃO

COMPARATIVO ARQUITETURAL: MONÓLITO, ARQUITETURA EM CAMADAS (MVC) E CLEAN ARCHITECTURE EM SISTEMAS ESCALÁVEIS

ARCHITECTURAL COMPARISON: MONOLITH, LAYERED ARCHITECTURE (MVC) AND CLEAN ARCHITECTURE IN SCALABLE SYSTEMS

Victor Hugo Geroto Toledo
ORCID: https://orcid.org/[preencher]
CPF: [preencher]
Centro Universitário SENAI São Paulo – UNISENAI-SP, Campus Sorocaba/SP
e-mail: [preencher]

Orientador: Prof. Me. Deivison Shindi Takatu
ORCID: https://orcid.org/[preencher]
CPF: [preencher]
Centro Universitário SENAI São Paulo – UNISENAI-SP, Campus Sorocaba/SP
e-mail: [preencher]

Orientador: Prof. Dr. Glauco Todesco
ORCID: https://orcid.org/[preencher]
CPF: [preencher]
Centro Universitário SENAI São Paulo – UNISENAI-SP, Campus Sorocaba/SP
e-mail: [preencher]

RESUMO: Este trabalho tem como objetivo comparar, de forma quantitativa e qualitativa, o desempenho, a integração e a manutenibilidade de sistemas de software estruturados em três padrões arquiteturais: Monólito, Arquitetura em Camadas (MVC) e Clean Architecture. A metodologia consiste em um protótipo tripartido, no qual a mesma aplicação de gerenciamento de tarefas é desenvolvida três vezes, mantendo constantes o domínio, a linguagem (TypeScript com Node.js), o banco de dados (PostgreSQL) e a infraestrutura em nuvem da Amazon Web Services (AWS), com carga controlada pelo Apache JMeter e monitoramento via Amazon CloudWatch e Grafana. A pesquisa está em andamento, em estágio de especificação arquitetural e planejamento experimental; a coleta de métricas ainda não foi executada e não há resultados quantitativos consolidados. Como resultados parciais, delimitou-se a lacuna da literatura, que compara majoritariamente monólitos e microsserviços, e formulou-se a hipótese de que a Clean Architecture apresentará melhor testabilidade e menor acoplamento, enquanto o Monólito terá menor latência em baixa carga. Espera-se produzir diretrizes baseadas em evidências para a seleção do padrão arquitetural conforme o contexto do projeto.
Palavras-chave: Arquitetura de Software; Computação em Nuvem; Desempenho; Escalabilidade; Manutenibilidade.

ABSTRACT: This work aims to compare, quantitatively and qualitatively, the performance, integration and maintainability of software systems structured in three architectural patterns: Monolith, Layered Architecture (MVC) and Clean Architecture. The methodology consists of a tripartite prototype, in which the same task management application is developed three times, keeping the domain, the language (TypeScript with Node.js), the database (PostgreSQL) and the Amazon Web Services (AWS) cloud infrastructure constant, with load controlled by Apache JMeter and monitoring via Amazon CloudWatch and Grafana. The research is ongoing, at the architectural specification and experimental planning stage; metrics collection has not yet been carried out and there are no consolidated quantitative results. As partial results, the gap in the literature, which mostly compares monoliths and microservices, was delimited, and the hypothesis was formulated that Clean Architecture will show better testability and lower coupling, while the Monolith will show lower latency under low load. The expected outcome is a set of evidence-based guidelines for selecting the architectural pattern according to the project context.
Keywords: Cloud Computing; Maintainability; Performance; Scalability; Software Architecture.

1 INTRODUÇÃO

A transformação digital ampliou a demanda sobre a arquitetura dos sistemas de software, que precisam sustentar grande volume de solicitações simultâneas e disponibilidade constante. A escolha do padrão arquitetural é uma decisão estratégica, pois afeta a evolução, a manutenção e a escalabilidade do sistema durante todo o seu ciclo de vida (Garlan; Shaw, 1993). Padrões como a Arquitetura em Camadas, difundida no modelo MVC (*Model-View-Controller*), e a Clean Architecture (Martin, 2017) favorecem a divisão de responsabilidades, a testabilidade e a independência de tecnologias externas.

Escolher o padrão que ofereça o melhor equilíbrio entre facilidade de implementação, manutenibilidade e escalabilidade em sistemas de médio porte é o problema central desta pesquisa. Essa escolha costuma se apoiar em preferências pessoais, modismos ou relatos anedóticos (Fowler, 2002), o que gera decisões subótimas, dívida técnica e custos crescentes de manutenção (Newman, 2015). Blinowski, Ojdowska e Przybylek (2022) mostraram que, em máquina única, o monólito supera os microsserviços em latência, o que reforça a necessidade de medição rigorosa. Poucos estudos, contudo, comparam os três modelos sob as mesmas condições operacionais.

O objetivo deste trabalho é analisar quantitativa e qualitativamente o desempenho, a integração e a manutenibilidade de sistemas construídos segundo os três padrões, por meio de um protótipo funcional tripartido hospedado na AWS, com resultados exibidos em um painel de métricas em tempo real.

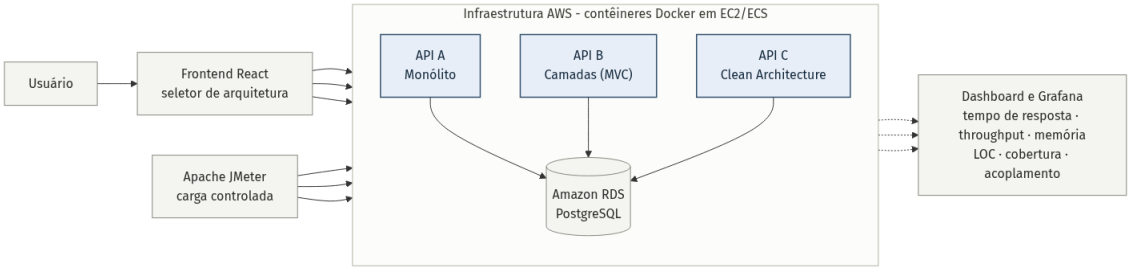
2 METODOLOGIA

A pesquisa adota a implementação de um protótipo tripartido: a mesma aplicação de gerenciamento de tarefas é desenvolvida três vezes, cada versão seguindo um padrão (Monólito, Arquitetura em Camadas/MVC e Clean Architecture). Assim, o padrão arquitetural é a variável independente, enquanto o domínio de negócio, a linguagem (TypeScript com Node.js), o banco de dados (PostgreSQL) e a infraestrutura (AWS) permanecem constantes. Cada versão contempla interface em React, API REST, regras de negócio para tarefas, autenticação por *JSON Web*

Tokens (JWT) e persistência em PostgreSQL, diferindo apenas na organização do código-fonte e na distribuição de responsabilidades.

Cada instância é implantada em contêineres Docker, em instâncias Amazon EC2 de configuração idêntica, orquestrados pelo Amazon Elastic Container Service (ECS), com dados no Amazon RDS (PostgreSQL), segundo o AWS Well-Architected Framework (Amazon Web Services, 2023). Tempo de resposta, *throughput* e consumo de memória são coletados sob carga controlada gerada pelo Apache JMeter, com monitoramento no Amazon CloudWatch e painéis no Grafana, o que permite comparar diretamente as três versões (Figura 1). Indicadores de qualidade de código, como cobertura de testes, dependências entre módulos e linhas de código por camada, complementam a análise.

Figura 1 – Arquitetura da solução e instrumentação da coleta de métricas



Fonte: Elaborado pelos autores (2026).

3 REFERENCIAL TEÓRICO

3.1 PADRÕES ARQUITETURAIS

A arquitetura de software estabelece a organização de um sistema por meio de decisões estruturais e princípios de projeto. Garlan e Shaw (1993) a definem como o estudo da organização global dos componentes, de suas interações e dos princípios que governam o projeto e a evolução do sistema; para Bass, Clements e Kazman (2012), é o conjunto de estruturas necessárias para raciocinar sobre o sistema, seus elementos e as relações entre eles. O modelo monolítico, que reúne interface, lógica de negócio e persistência em um único artefato implantável, é historicamente o mais utilizado: proporciona baixa latência em contextos de baixa complexidade, mas limita o escalonamento seletivo e amplia o acoplamento à medida que o sistema cresce (Dragoni *et al.*, 2017).

A Arquitetura em Camadas, popularizada pelo MVC, reduz o acoplamento por meio da separação horizontal entre apresentação, lógica de negócio e persistência,

melhorando a organização do código e facilitando a escalabilidade (Tu, 2023). A Clean Architecture (Martin, 2017) isola as regras de negócio no núcleo de camadas concêntricas e aplica o princípio da inversão de dependências, o que favorece a testabilidade e a independência tecnológica.

3.2 TRABALHOS RELACIONADOS

A comparação empírica entre padrões arquiteturais é uma linha de investigação estabelecida, porém concentrada no eixo monólito–microsserviços. O trabalho metodologicamente mais próximo é o de Blinowski, Ojdowska e Przybylek (2022), que compararam, de forma controlada, uma aplicação monolítica e sua versão em microsserviços, mantendo constantes a funcionalidade e o ambiente de execução. Em máquina única, a versão monolítica apresentou menor latência, atribuída à comunicação em processo, enquanto a distribuída foi superior no escalonamento horizontal sob carga crescente. Esse achado fundamenta a adoção das mesmas métricas (tempo de resposta, *throughput* e consumo de memória) e a hipótese de menor latência do monólito em baixa carga. Tapia *et al.* (2020) seguiram desenho semelhante, com a mesma funcionalidade sob arquiteturas distintas, medida em condições equivalentes, com foco em consumo de recursos de hardware e latência, procedimento que este trabalho replica.

Revisões sistemáticas recentes reforçam essa leitura: Di Francesco, Lago e Malavolta (2019) mapearam 103 estudos primários sobre arquitetura de microsserviços e evidenciaram a predominância desse eixo; Abgaz *et al.* (2023) analisaram 35 estudos sobre decomposição de monólitos e concluíram que o tema permanece em estágio inicial de maturidade; e Velepucha e Flores (2023) sistematizaram princípios e desafios de migração. A literatura compara majoritariamente monólitos e microsserviços, sem considerar a Clean Architecture como alternativa estruturalmente coesa (Richards, 2015). Nenhum dos estudos identificados a avalia empiricamente em comparação controlada com o monólito e com a Arquitetura em Camadas, sob identidade funcional, de linguagem e de infraestrutura, lacuna em que se insere este trabalho.

4 RESULTADOS E DISCUSSÃO

A pesquisa está em desenvolvimento. Foram concluídas a revisão de literatura, a definição da arquitetura proposta e a especificação técnica do protótipo, incluindo a pilha tecnológica (TypeScript/Node.js, React, PostgreSQL) e os serviços AWS (EC2, ECS, RDS, S3, IAM e CloudWatch). A implementação das três versões e a coleta de métricas ainda não foram concluídas; portanto, não há dados quantitativos consolidados para as três arquiteturas.

Com base na literatura, formulou-se a hipótese que orientará a análise (Quadro 1): a Clean Architecture deve apresentar melhor testabilidade e menor acoplamento entre módulos; o Monólito, menor latência em baixa carga, replicando o resultado de Blinowski, Ojdowska e Przybylek (2022); e a Arquitetura em Camadas, o melhor equilíbrio entre simplicidade e organização para equipes menores. O Quadro 1 expressa expectativas derivadas da literatura, e não medições; elas serão confrontadas com os dados da fase experimental, sob condições controladas que eliminam variáveis externas.

Quadro 1 – Hipóteses de comportamento relativo entre os padrões avaliados

Critério	Monólito	Camadas (MVC)	Clean Architecture
Latência em baixa carga	Menor	Intermediária	Maior
Acoplamento entre módulos	Alto	Intermediário	Baixo
Testabilidade	Baixa	Intermediária	Alta
Complexidade de implementação	Baixa	Intermediária	Alta

Fonte: Elaborado pelos autores (2026), a partir da literatura revisada. Expectativas a verificar; não são resultados medidos.

5 CONSIDERAÇÕES FINAIS

Em relação ao objetivo proposto, este trabalho estabeleceu o desenho de um estudo comparativo entre Monólito, Arquitetura em Camadas (MVC) e Clean Architecture, delimitando a lacuna da literatura e o protocolo experimental que isola o padrão arquitetural como única variável independente. Ainda não é possível afirmar qual padrão é mais adequado a cada contexto, pois isso depende de dados que ainda serão coletados.

As próximas etapas são a implementação das três versões e a execução dos testes de carga, que permitirão obter métricas de desempenho, qualidade de código e manutenibilidade sob condições controladas. Como trabalhos futuros, propõem-se

pipelines de integração e entrega contínua (CI/CD) para cada versão e a investigação da decomposição da Clean Architecture em microsserviços.

REFERÊNCIAS BIBLIOGRÁFICAS

ABGAZ, Y. *et al.* Decomposition of monolith applications into microservices architectures: a systematic review. **IEEE Transactions on Software Engineering**, v. 49, n. 8, p. 4213-4242, 2023. DOI: 10.1109/TSE.2023.3287297.

AMAZON WEB SERVICES. **AWS Well-Architected Framework**. [S. l.]: AWS, 2023. Disponível em: https://aws.amazon.com/architecture/well-architected/. Acesso em: 20 maio 2026.

BASS, L.; CLEMENTS, P.; KAZMAN, R. **Software architecture in practice**. 3. ed. Boston: Addison-Wesley, 2012.

BLINOWSKI, G.; OJDOWSKA, A.; PRZYBYŁEK, A. Monolithic vs. microservice architecture: a performance and scalability evaluation. **IEEE Access**, v. 10, p. 20357-20374, 2022. DOI: 10.1109/ACCESS.2022.3152803.

DI FRANCESCO, P.; LAGO, P.; MALAVOLTA, I. Architecting with microservices: a systematic mapping study. **Journal of Systems and Software**, v. 150, p. 77-97, 2019. DOI: 10.1016/j.jss.2019.01.001.

DRAGONI, N. *et al.* Microservices: yesterday, today, and tomorrow. *In*: MAZZARA, M.; MEYER, B. (ed.). **Present and ulterior software engineering**. Cham: Springer, 2017. p. 195-216.

FOWLER, M. **Patterns of enterprise application architecture**. Boston: Addison-Wesley, 2002.

GARLAN, D.; SHAW, M. An introduction to software architecture. *In*: AMBRIOLA, V.; TORTORA, G. (ed.). **Advances in software engineering and knowledge engineering**. Singapore: World Scientific Publishing, 1993. p. 1-39.

MARTIN, R. C. **Clean architecture**: a craftsman's guide to software structure and design. Upper Saddle River: Prentice Hall, 2017.

NEWMAN, S. **Building microservices**: designing fine-grained systems. Sebastopol: O'Reilly Media, 2015.

RICHARDS, M. **Software architecture patterns**. Sebastopol: O'Reilly Media, 2015.

TAPIA, F. *et al.* From monolithic systems to microservices: a comparative study of performance. **Applied Sciences**, v. 10, n. 17, art. 5797, 2020. DOI: 10.3390/app10175797.

TU, Z. Research on the application of layered architecture in computer software development. **Journal of Computing and Electronic Information Management**, v. 11, n. 3, p. 34-38, 2023. DOI: 10.54097/jceim.v11i3.08.

VELEPUCHA, V.; FLORES, P. A survey on microservices architecture: principles, patterns and migration challenges. **IEEE Access**, v. 11, p. 88339-88358, 2023. DOI: 10.1109/ACCESS.2023.3305687.